COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CpE21S3
DATE PERFORMED:	09/15/2026	**DATE SUBMITTED:**	09/15/2026
NAME:	Wayne Kaisen C. Paraiso	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 8.1 Sorting Algorithms Pt2

1. PROCEDURE OUTPUT

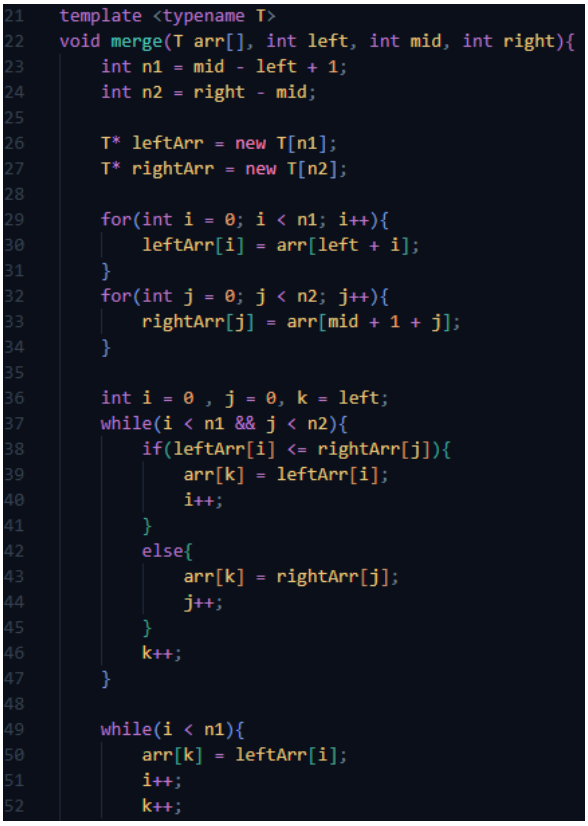
Code + Console Screenshot	
Observations	The code initializes a maxSize-based array pair, seeds rand() with the current time to randomly generate 100 values (0-99) into unsortedArr, and maintains a copy of the array as an untouched duplicate in copiedArr for later use by multiple sort algorithms attempting to sort the same unsorted data; a reusable displayAll() is then used to display the array as it is generated following initialization, serving as the unsorted baseline prior to any sorting operations taking place.

Array of Values for Sort Algorithm Testing

Code + Console Screenshot	

Observations	The shellSort function is templated with T, allowing it to sort any data type, and it sorts by starting with a large gap (half the array size) and halving it with each pass until it reaches 0, doing a gapped insertion sort at each pass, which saves an element in temp and moves the elements it is larger than spaced gap apart forward and then places the temp once its correct place is found, which allows the elements to move farther per swap early on and requires less total shifts than a regular insertion sort.

Shell Sort Technique

Code + Console Screenshot	

```
template <typename T>
void merge(T arr[], int left, int mid, int right){
    int n1 = mid - left + 1;
    int n2 = right - mid;

    T* leftArr = new T[n1];
    T* rightArr = new T[n2];

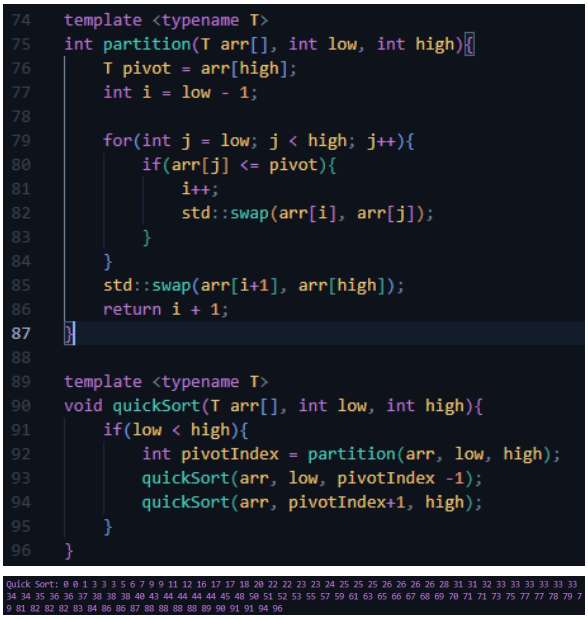
    for(int i = 0; i < n1; i++){
        leftArr[i] = arr[left + i];
    }
    for(int j = 0; j < n2; j++){
        rightArr[j] = arr[mid + 1 + j];
    }

    int i = 0 , j = 0, k = left;
    while(i < n1 && j < n2){
        if(leftArr[i] <= rightArr[j]){
            arr[k] = leftArr[i];
            i++;
        }
        else{
            arr[k] = rightArr[j];
            j++;
        }
        k++;
    }

    while(i < n1){
        arr[k] = leftArr[i];
        i++;
        k++;
```

	
Observations	The templated merge function divides arr[left..right] into two temporary subarrays (leftArr, rightArr) at mid, and merges them back together by continually copying the smaller front element into arr, followed by two loops that copy any remaining elements from either side; mergeSort then proceeds this recursively by dividing the array at the midpoint, then sorting each half by recursively calling, and then merging the result, until left < right is no longer true.

Merge Sort Algorithm

Code + Console Screenshot	
Observations	The templated partition function selects the last element of the array, arr[high], as the pivot and proceeds to iterate through the array using j, swapping any element less than or equal to the pivot to the left partition of the array and tracked by i, and finally, swapping the pivot into its rightful place before returning the index of it; quickSort subsequently utilizes this pivot partitioning to recursively sort subarrays on either side of the returned index until every subarray call has low >= high, signifying a sorted array.

Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1:
Right, so once you've done the partition, then the array is already partially sorted, because the pivot is in its correct position, with smaller elements to its left and larger ones to its right. That means you have two independent subarrays that need to be sorted, which you can sort by recursively calling quicksort on them. It's also possible to use a different sorting algorithm, because the only requirement is that you sort them in place, and you can choose whatever algorithm you want for that, as long as you meet the requirement. Your code does that by choosing insertion sort for the left and right subarrays:
suppMain.cpp

supp > suppMain.cpp > main()

```cpp
#include <iostream>
#include "suppSort.h"

void displayArr(int arr[], int size);

int main(){
    int testArr[] = {66, 89, 45, 76, 22, 10, 16}; int size = sizeof(testArr) / sizeof(testArr[0]);

    std::cout << "Before quickSortHybrid:\n";
    displayArr(testArr, size);

    quickSortHybrid(testArr, 0, size - 1);

    std::cout << "After quickSortHybrid:\n";
    displayArr(testArr, size);

    return 0;
}

void displayArr(int arr[], int size){
    for(int i = 0; i < size; i++){
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}
```

suppSort.h

```cpp
#ifndef SUPPSORT_H
#define SUPPSORT_H

#include <utility>

template <typename T>
void insertionSort(T arr[], int arrSize){
    int k = 1, j;
    T temp;

    while(k <arrSize){
        temp = arr[k];
        j = k - 1;

        while(j >= 0 && temp < arr[j]){
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j+1] = temp;
        k++;
    }
}

template <typename T>
int partition(T arr[], int low, int high){
    T pivot = arr[high];
    int i = low- 1;

    for(int j = low; j < high; j++){
        if(arr[j] <= pivot){
```

```
            i++;
            std::swap(arr[i], arr[j]);
        }
    }

    std::swap(arr[i + 1], arr[high]);
    return i + 1;
}

template <typename T>
void quickSortHybrid(T arr[], int low, int high){
    if(low < high){
        int pivotIndex = partition(arr, low, high);
        insertionSort(arr + low, pivotIndex - low);
        insertionSort(arr + pivotIndex + 1, high - pivotIndex);
    }
}

#endif
```

```
Before quickSortHybrid:
66 89 45 76 22 10 16
After quickSortHybrid:
10 16 22 45 66 76 89
```

Problem 2:

For this 15-element array, insertion sort would give the fastest actual run-time performance despite its $O(n^2)$ complexity because at this small scale the overhead of the O(n log n) approach is worse (no recursion, no allocation, just comparisons and swaps), so its asymptotic running-time advantage only pays off for larger n (typically somewhere in the 20–50+ range, depending on implementation details) and outweighs the $O(n^2)$ for lower n. Merge sort and quicksort both achieve O(n log n) because they split the array in halves that are then merge-sorted or quick-sorted again, which can only happen $\log_2 n$ times before the sub-arrays are of size 1, and at each level of recursion the algorithm performs O(n) work (merging for merge-sort, partitioning for quicksort), so overall O(n) × O(log n) = O(n log n); their difference lies in the fact that merge sort always does an even split (providing the asymptotic guarantee for all cases), while quicksort's partitions can be uneven, giving it an average but not worst-case O(n log n) performance.

Conclusion:
This activity provided hands-on experience implementing and comparing shell sort, merge sort, and quick sort, demonstrating how each algorithm approaches the same goal of ordering data through fundamentally different strategies: shell sort improves on basic insertion sort by first arranging elements into gapped sub-lists to reduce the number of shifts needed before a final insertion sort pass, merge sort applies a divide-and-conquer approach that recursively splits the array down to individual elements and then merges them back together in sorted order, and quick sort also divides the array but does so by partitioning elements around a pivot so that smaller elements fall to one side and larger elements to the other before recursively sorting each partition. Through implementing these algorithms in C++ and testing them on a randomized 100-element array, it becomes clear that no single algorithm is universally "best"; rather, each has trade-offs in time complexity, memory usage, and stability depending on the size and nature of the input data, reinforcing the practical importance of understanding not just how to code a sorting algorithm, but when and why to choose one over another.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar Correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I have a draft for the observations and conclusions and let claude fix the grammatical errors**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.